

СПбГУ, ИИНоД, 1 курс, 2026/27

Алгоритмы

Конспект лекций 1-2

Иван Пальченков

Лекция 1. Асимптотика

1.1. Какой алгоритм считать удачным

Алгоритм для нас — это ясно описанный способ получить ответ по входу. Удачным его делают четыре свойства.

Он обязан быть верным на всех допустимых входах. Частный пример, на котором программа «обычно работает», алгоритмом курса не считается.

Он должен быть быстрым. Время зависит от размера входа. Этот размер чаще всего обозначают n . Линейный просмотр массива делает на каждый элемент постоянное число элементарных шагов, поэтому время растёт пропорционально n . Сначала смотрят на эту зависимость, и только потом на множитель перед ней: при больших n множитель проигрывает более крутому росту.

Ориентир по абсолютному времени — около секунды. Одно ядро современного процессора выполняет порядка 10^9 простых команд в секунду, если писать на C++. На Python за ту же секунду реально успевает порядка 10^6 шагов интерпретатора. Минута для интерактивной программы уже ощущается долго, сотая доля секунды человеком не различается.

Память — отдельный ресурс, и часто более дорогой, чем лишний проход по уже лежащим в кеше данным. Об этом речь на второй лекции.

Наконец, алгоритм должен быть коротким и без россыпи особых случаев. То, что трудно пересказать, трудно и реализовать без ошибки.

Дальше в этой лекции считаем все элементарные операции одинаково дорогими и следим только за тем, как число шагов растёт с n . Функцию времени обозначаем $f(n)$. Считаем n натуральным, а $f(n)$ положительной.

1.2. Обозначения O , Θ , Ω , o , ω

Пусть f и g переводят натуральные числа в положительные вещественные.

Определение.

$$f = \Theta(g) \quad \exists N > 0, C_1 > 0, C_2 > 0: \forall n \geq N, C_1 \cdot g(n) \leq f(n) \leq C_2 \cdot g(n)$$

Определение.

$$f = O(g) \quad \exists N > 0, C > 0: \forall n \geq N, f(n) \leq C \cdot g(n)$$

Определение.

$$f = \Omega(g) \quad \exists N > 0, C > 0: \forall n \geq N, f(n) \geq C \cdot g(n)$$

Определение.

$$f = o(g) \quad \forall C > 0 \exists N > 0: \forall n \geq N, f(n) \leq C \cdot g(n)$$

Определение.

$$f = \omega(g) \quad \forall C > 0 \exists N > 0: \forall n \geq N, f(n) \geq C \cdot g(n)$$

Смысл Θ — «равны с точностью до константы». Смысл O — «не больше с точностью до константы». Смысл o — «меньше сколь угодно малого постоянного множителя»: константу нельзя зафиксировать заранее, она обязана работать для любого C , даже

очень маленького.

Θ	O	Ω	o	ω
$=$	$\leq$	$\geq$	$<$	$>$

Аналогия помогает помнить направление, но кванторы разные: у O константа существует, у o она произвольна.

Свойства.

1. $f = \Theta(g)$ тогда и только тогда, когда $g = \Theta(f)$.
2. $f = O(g)$ и $g = O(f)$ тогда и только тогда, когда $f = \Theta(g)$.
3. $f = \Omega(g)$ тогда и только тогда, когда $g = O(f)$.
4. $f = \omega(g)$ тогда и только тогда, когда $g = o(f)$.
5. Каждое из пяти отношений транзитивно.
6. Для любой константы $C > 0$ верно $C \cdot f = \Theta(f)$.

Первое свойство получается переверотом неравенства.

$$C_1 g(n) \leq f(n) \leq C_2 g(n) \Rightarrow \frac{1}{C_2} f(n) \leq g(n) \leq \frac{1}{C_1} f(n)$$

Лемма. Если $g = o(f)$, то $f + g = \Theta(f)$ и $f - g = \Theta(f)$, как только разность остаётся положительной.

Доказательство. Возьмём $C = 1/2$. С некоторого места $g \leq f/2$, поэтому

$$\frac{1}{2}f(n) \leq f(n) \pm g(n) \leq \frac{3}{2}f(n)$$

Лемма. При любом фиксированном $k \geq 0$

$$n^k = o(n^{k+1})$$

Для любого $C > 0$ при $n \geq C$ выполнено $n^{k+1} \geq C \cdot n^k$.

Лемма. Многочлен с положительным старшим коэффициентом и степенью d есть $\Theta(n^d)$.

Все младшие степени есть o от старшей, их поглощает предыдущая лемма. Константа при старшем члене не меняет Θ .

1.3. Умножение и Карацуба

Произведение двух многочленов, заданных массивами коэффициентов, считается двойным циклом: в ячейку $c[i+j]$ добавляется $a[i] \cdot b[j]$. Это $\Theta(nm)$ умножений. Длинные целые в записи по фиксированному основанию умножаются так же, после чего отдельным проходом разносятся переносы. Переносы линейны по длине результата. Значит, умножение двух n -значных чисел школьным способом стоит $\Theta(n^2)$, и достаточно уметь умножать многочлены.

Карацуба уменьшает число рекурсивных умножений с четырёх до трёх. Пусть длина n чётная, $k = n/2$, и половины содержат по k коэффициентов.

$$A = A_0 + x^k A_1, \quad B = B_0 + x^k B_1$$

$$AB = A_0 B_0 + x^k (A_0 B_1 + A_1 B_0) + x^{2k} A_1 B_1$$

Четыре произведения дают рекуррентность

$$T_4(n) = 4T_4(n/2) + \Theta(n)$$

Среднее произведение собирается из уже посчитанных:

$$A_0 B_1 + A_1 B_0 = (A_0 + A_1)(B_0 + B_1) - A_0 B_0 - A_1 B_1$$

Остаются три умножения:

$$T(n) = 3T(n/2) + \Theta(n)$$

Сложение половин и сдвиги массивов и есть добавка $\Theta(n)$.

```

1 Poly karatsuba(Poly A, Poly B) { // длины равны и являются степенью двойки
2     int n = A.size();
3     if (n == 1)
4         return Poly{A[0] * B[0]};
5     int k = n / 2;
6     Poly A0 = half(A, 0, k), A1 = half(A, k, n);
7     Poly B0 = half(B, 0, k), B1 = half(B, k, n);
8     Poly P0 = karatsuba(A0, B0);
9     Poly P2 = karatsuba(A1, B1);
10    Poly P1 = karatsuba(add(A0, A1), add(B0, B1));
11    Poly mid = sub(sub(P1, P0), P2); // A0*B1 + A1*B0
12    return add(P0, shift(mid, k), shift(P2, 2 * k));
13 }
```

Сумма половин может оказаться на один коэффициент длиннее. Это константный случай внутри рекурсии и асимптотику не ломает. Для чисел после умножения «как многочленов» отдельно выполняют переносы.

1.4. Мастер-теорема

Теорема. Пусть $a > 0$, $b > 1$, $c \geq 0$ и n — степень b . Положим $k = \log_b n$.

$$T(n) = aT(n/b) + n^c$$

Тогда

$$T(n) = \Theta(n^{\log_b a}), \quad a > b^c$$

$$T(n) = \Theta(n^c), \quad a < b^c$$

$$T(n) = \Theta(n^c \log n), \quad a = b^c$$

Доказательство. Раскрываем дерево рекурсии:

$$T(n) = n^c + a(n/b)^c + a^2(n/b^2)^c + \dots + a^k(n/b^k)^c$$

$$T(n) = n^c (1 + q + q^2 + \dots + q^k), \quad q = a/b^c$$

В сумме $k+1$ слагаемых. При $q = 1$ сумма равна $\Theta(\log n)$. При $q < 1$ сумма геометрической прогрессии ограничена константой, зависящей только от q . При $q > 1$ сумма есть $\Theta(q^k)$, и поскольку $n = b^k$,

$$n^c q^k = n^c \cdot a^k / b^{ck} = a^k = n^{\log_b a}$$

Для Карацубы $a = 3$, $b = 2$, $c = 1$. Так как $3 > 2$, получаем $\Theta(n^{\log_2 3})$, и $\log_2 3 \approx 1.585$. Школьное умножение в той же форме — это $T(n) = 4T(n/2) + \Theta(n)$. Здесь $4 > 2$, поэтому $\Theta(n^2)$. Мастер-теорема совпадает с прямым подсчётом двойного цикла.

Обобщение. Если добавка равна $n^c \log^d n$ при $d \geq 0$, три случая сохраняются:

$$a > b^c \Rightarrow T(n) = \Theta(n^{\log_b a})$$

$$a < b^c \Rightarrow T(n) = \Theta(n^c \log^d n)$$

$$a = b^c \Rightarrow T(n) = \Theta(n^c \log^{d+1} n)$$

Подсчёт тот же: на каждом уровне появляется лишний логарифм, а при $q = 1$ логарифмы складываются в ещё одну степень.

1.5. Линейные рекуррентности

Теорема. Пусть все $a_i > 0$, все $b_i > 0$ и сумма b_i больше 1.

$$T(n) = \sum_i b_i T(n - a_i)$$

Тогда $T(n) = \Theta(\alpha^n)$, где $\alpha > 1$ — единственный корень уравнения

$$1 = \sum_i b_i \alpha^{-a_i}$$

Корень ищут бинарным поиском на $[1, +\infty)$.

Подстановка $T(n) = \alpha^n$ сразу даёт это уравнение. Правая часть как функция α строго убывает, в единице она больше 1, а на бесконечности стремится к 0. Значит, корень ровно один.

Оценка сверху доказывается индукцией. На начальном отрезке длины $\max a_i$ выбираем константу C так, чтобы $T(n) \leq C \alpha^n$. Шаг:

$$T(n) = \sum_i b_i T(n - a_i) \leq C \sum_i b_i \alpha^{n - a_i} = C \alpha^n$$

Последнее равенство — уравнение на α . Нижняя оценка пишется теми же выкладками с обратным неравенством и своей константой.

1.6. Угадать и доказать индукцией

Часто вид ответа угадывают, а индукция только проверяет константу.

Пример. $T(1) = 1$ и $T(n) = T(\lfloor n/2 \rfloor) + n$. Утверждение: $T(n) \leq 2n$. База: $1 \leq 2$. Переход:

$$T(n) \leq 2\lfloor n/2 \rfloor + n \leq n + n = 2n$$

Если угаданная константа не проходит, её увеличивают. Если не проходит никакая, угадана не та степень: неравенство не замыкается.

Ещё три коротких образца.

1. $T(n) = 2T(n-1) + T(n-2)$. Подстановка 2^n не обязана быть корнем. Надёжнее характеристическое уравнение из пункта 1.5.
2. $T(n) = 2T(n-3)$. Раскрытие даёт $T(n) = \Theta(2^{n/3})$.
3. $T(n) = T(n-1) + T(n-3)$. Уравнение $1 = \alpha^{-1} + \alpha^{-3}$, корень $\alpha \approx 1.465$, и $T(n) = \Theta(\alpha^n)$.

1.7. Числа Фибоначчи

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2}$$

Сдвиг нумерации, при котором оба первых члена равны 1, на асимптотику не влияет.

Сверху $F_n \leq 2F_{n-1}$, поэтому $F_n \leq 2^n$. Снизу $F_n \geq 2F_{n-2}$, поэтому $F_n \geq 2^{n/2}$ с поправкой на базу. Теорема пункта 1.5 уточняет показатель:

$$1 = \alpha^{-1} + \alpha^{-2} \Leftrightarrow \alpha^2 - \alpha - 1 = 0$$

$$\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618, \quad F_n = \Theta(\varphi^n)$$

1.8. Те же обозначения через пределы

Для положительных функций $f = o(g)$ тогда и только тогда, когда $f(n)/g(n) \rightarrow 0$ при $n \rightarrow \infty$. Это ровно квантор «для всякого $C > 0$ начиная с некоторого места отношение не больше C ».

Запись $f = O(g)$ означает, что отношение f/g ограничено на хвосте. Если обычный предел существует и конечен, этого достаточно. Если предел колеблется, смотрят на верхний предел отношения: он должен быть конечен.

1.9. Суммы и интегралы

Определённый интеграл положительной функции — площадь под графиком.

Лемма. Если f возрастает на отрезке длины 1, то

$$f(a) \leq \int_a^{a+1} f(x) dx \leq f(a+1)$$

Складывая такие неравенства, для возрастающей положительной f получают

$$\int_a^b f \leq \sum_{i=a}^b f(i) \leq f(b) + \int_a^b f$$

Для убывающей положительной функции интеграл по-прежнему зажат между суммами, только выброшенный край — это $f(a)$, а не $f(b)$.

Следствие. Если f возрастает и положительна, а интегралы от 1 до n и от 1 до $n+1$ отличаются не больше чем в константу раз, то сумма значений $f(1)+\dots+f(n)$ есть Θ от этого интеграла.

Формула Ньютона–Лейбница и гармонический интеграл:

$$\int_a^b f'(x) dx = f(b) - f(a), \quad \int_1^n \frac{dx}{x} = \ln n$$

Гармоническую сумму можно оценить и без интеграла, группируя степени двойки. Сверху каждый блок от 2^t до $2^{t+1}-1$ состоит из не больше чем 2^t слагаемых, каждое не больше $1/2^t$, и блок не больше 1. Таких блоков $O(\log n)$. Снизу соседний блок даёт не меньше $1/2$. Поэтому

$$\sum_{k=1}^n \frac{1}{k} = \Theta(\log n)$$

1.10. Несколько прямых подсчётов

Пять вложенных циклов, в которых каждый следующий индекс строго меньше предыдущего, увеличивают счётчик ровно $C(n, 5)$ раз. Это $\Theta(n^5)$, но константа $1/5!$ в грубой оценке $O(n^5)$ не видна. При небольшом n факториал в знаменателе решает, уложится ли программа в секунду.

n -е число Фибоначчи циклом из n сложений считается не за $\Theta(n)$, если складывать длинные числа честно. Длина F_n есть $\Theta(n)$ бит, сложение i -го члена стоит $\Theta(i)$, сумма по i есть $\Theta(n^2)$.

Список всех делителей всех чисел от 1 до n строится так: для каждого a пройти кратные $b = a, 2a, 3a, \dots$ и дописать a в список делителей b . Число итераций — n раз гармоническая сумма, то есть $\Theta(n \log n)$.

Представление $N = a^2 + b^2$ не обязано стоить $\Theta(N)$. Положим b равным $\lfloor \sqrt{N} \rfloor$ и увеличиваем a . Пока сумма квадратов слишком велика, уменьшаем b . Переменная a растёт не больше $\sqrt{N}$ раз, переменная b убывает не больше $\sqrt{N}$ раз, поэтому внутренний цикл за всё время делает $O(\sqrt{N})$ шагов, хотя выглядит вложенным. Это первый образец двух указателей: каждый двигается только в одну сторону.

1.11. Что растёт быстрее

Линейное время — $O(n)$. Квадратичное — $O(n^2)$. Полиномиальное — $O(n^k)$ при каком-то фиксированном k . Полилогарифм — $O(\log^k n)$. Экспонента в узком смысле этой лекции — $O(z^n)$ при каком-то $z > 1$.

Теорема. Для любых $x, y > 0$ и $z > 1$ при всех достаточно больших n

$$\log^x n < n^y < z^n$$

Сначала $n^2 < 2^n$ при $n \geq 10$: на отрезке $[10, 20)$ это проверка руками. Если неравенство верно для n , то для $2n$ левая часть растёт в 4 раза, правая — в 2^n раз, и при $n \geq 2$ этот множитель не меньше 4. Любое большое n попадает в отрезок, полученный из $[10, 20)$ несколькими удвоениями.

Сравнение n^y и z^n сводится к предыдущему заменой переменной. Логарифм против степени сводится к тому же, если положить $k = \log n$.

Отсюда логарифм есть o от любой положительной степени n , а любая степень есть o от показательной функции с основанием больше 1. Чтобы получить именно o , а не только O , теорему применяют к чуть меньшим показателям: лишний множитель уходит в ноль.

Лекция 2. Структуры данных

2.1. Чтобы код структур вообще запускался

Включайте предупреждения компилятора и не оставляйте их в сборке.

```
1 // g++ и clang
2 // -Wall -Wextra -Wshadow
```

Неопределённое поведение — ситуация, в которой язык не обещает ничего: программа может упасть, может молча посчитать не то, может вести себя по-разному на двух машинах. Самые частые источники: нет `return` из функции, чтение неинициализированной переменной, выход за границу массива или вектора, испорченный итератор.

У массива `int a[4]` индексы 0, 1, 2, 3. Запись в `a[4]` — уже неопределённое поведение. Вектор в отладочном режиме стандартной библиотеки такую запись ловит, если соответствующий макрос стоит до подключения заголовков.

```
1 #define _GLIBCXX_DEBUG // до любых #include
2 #include <vector>
3 std::vector<int> a(4);
4 a[4] = 7; // поймается в отладочной сборке
```

Структура собирает несколько полей в одно значение. Указатель хранит адрес. Если `p` имеет тип «указатель на `int`», то `*p` — число по этому адресу, а `&x` — адрес переменной `x`. Имя массива в выражении превращается в указатель на нулевой элемент. Запись `p->x` означает то же, что `(*p).x`.

```
1 struct Point { int x, y; };
2
3 Point q{2, 3};
4 int value = 3;
5 int* ptr = &value;
6 *ptr = 7; // value стало 7
7
8 Point* heap = new Point{0, 0};
9 heap->x = 3;
10 delete heap;
```

2.2. Что дорого, хотя в Θ этого не видно

Асимптотика считает шаг единицей. На машине единицы разные.

1. Ввод и вывод. Посимвольный обмен с потоком гораздо медленнее чтения кусками в буфер.
2. Квадратный корень, синус, арктангенс. Это не одна команда на все случаи: аргумент приближают рядом или итерациями.
3. Деление и остаток дороже сложения и сдвига.
4. Доступ к памяти. Сумма `a[i]` читает соседние ячейки: одна загрузка кеш-линии обслуживает несколько итераций. Сумма `a[r[i]]` при случайной перестановке

прыгает по адресам.

5. Выделение и освобождение. `new` и `delete` ходят в аллокатор.

6. Вызов функции. Глубокая рекурсия из n пустых вызовов при отключённой оптимизации держит $\Theta(n)$ кадров стека.

```
1 void go(int n) {
2     if (n <= 0) return;
3     go(n - 1);    // без оптимизации это n кадров стека
4 }
```

Память слоями. Диск — самый большой и самый медленный. Оперативная память — гигабайты. Кеш последнего уровня — единицы мегабайт. Ближний кеш ядра — десятки килобайт. Два алгоритма с одним и тем же $\Theta(n^2)$ шагов ведут себя по-разному, если первый держит таблицу $\Theta(n^2)$, а второй — массив длины $\Theta(n)$. Первый постоянно ходит в оперативную память. Таблица из четырёхбайтовых чисел порядка нескольких десятков тысяч строк уже не помещается в типичные 8 ГБ. Отдельно: алгоритм за $\Theta(2^n)$ времени и $\Theta(2^n)$ памяти при n около 40 не запускается из-за памяти, а тот же по времени алгоритм с полиномиальной памятью ещё может досчитать.

Копирование большого куска памяти библиотекой быстрее рукописного цикла: процессор двигает сразу 16 или 32 байта. Поэтому плотный массив обгоняет россыпь мелких объектов.

2.3. Префиксные суммы

Нужно много раз считать сумму на отрезке $[l, r]$. Наивно каждый запрос стоит $\Theta(r - l + 1)$. Если запросов порядка n и отрезки длинные, набегают $\Theta(n^2)$.

```
1 void precalc() {                // один проход,  $\Theta(n)$ 
2     sum[0] = 0;
3     for (int i = 0; i < n; i++)
4         sum[i + 1] = sum[i] + a[i];
5 }
6 int get(int l, int r) {         // сумма  $a[l] + \dots + a[r]$ 
7     return sum[r + 1] - sum[l]; //  $\Theta(1)$ 
8 }
```

Сумма всего массива — это `sum[n]`. Сумма одного элемента `a[i]` — это `sum[i + 1] - sum[i]`.

Одно присваивание `a[i] = x` портит все префиксы правее `i`. Если обновления и запросы чередуются по одному, пересчёт всего массива `sum` на каждое обновление снова квадратичен. Префикс удобен, когда массив сначала собрали, а потом только читают, либо когда обновления идут пачкой до серии запросов.

2.4. Массив

Массив фиксированной длины даёт чтение и запись по индексу за $\Theta(1)$. Поиск значения — $\Theta(n)$. Если занято ровно n ячеек и длина хранится отдельно, удаление последнего элемента — $\Theta(1)$: достаточно уменьшить длину. Удаление первого — $\Theta(n)$, потому что хвост сдвигается. Добавление в начало или в конец при каждом изменении размера заставляет выделить новый кусок и скопировать все элементы. Если запас ячеек в конце уже есть, добавление в конец дешевле до $\Theta(1)$, добавление в начало по-прежнему сдвигает данные.

2.5. Двусвязный список

Узел хранит значение и два указателя. У списка есть фиктивные голова и хвост:

настоящие элементы лежат строго между ними. Пустой список — это пара ссылок «голова смотрит на хвост, хвост смотрит на голову».

```

1  struct Node {
2      Node* prev;
3      Node* next;
4      int x;
5  };
6  struct List {
7      Node* head;    // фиктивная голова
8      Node* tail;    // фиктивный хвост
9  };
10
11 void make_empty(List& L) {
12     L.head = new Node();
13     L.tail = new Node();
14     L.head->next = L.tail;
15     L.tail->prev = L.head;
16 }
17
18 void erase(Node* node) {
19     node->prev->next = node->next;
20     node->next->prev = node->prev;
21     delete node;
22 }
```

Доступ по индексу стоит $\Theta(i)$. Поиск значения — $\Theta(n)$. Оба конца и удаление по уже известному указателю — $\Theta(1)$. Хвост нужен, чтобы не искать конец. Ссылка назад нужна, чтобы при удалении починить левого соседа. Фиктивные узлы убирают отдельные ветки «удаляем первый» и «удаляем последний».

2.6. Односвязный список

Хранится только указатель на первый узел. У узла одно поле `next`. Вставка и удаление спереди — $\Theta(1)$.

```

1  struct Node { Node* next; int x; };
2
3  void push_front(Node*& head, int x) {
4      Node* node = new Node();
5      node->x = x;
6      node->next = head;
7      head = node;
8  }
```

Вставка в конец без отдельного хвоста — $\Theta(n)$. Удаление последнего хочет ещё и предпоследний узел. Удаление из середины по указателю на сам узел за $\Theta(1)$ невозможно: предыдущий неизвестен. Если предыдущий уже дан, хватает одной ссылки вперёд.

2.7. Список индексами в массиве

Указатель можно заменить номером ячейки. Поле `next` — индекс следующего узла, отдельное значение означает конец. Голова списка — тоже индекс.

```

1  struct Node { int next; int x; };
2  vector<Node> pool;
3  int head = -1;
4
```

```

5 void push_front(int x) {
6     pool.push_back(Node{head, x});
7     head = (int)pool.size() - 1;
8 }

```

Узлы лежат плотно, аллокатор вызывается редко, обход дружелюбнее к кешу, чем россыпь отдельных new. Если завести и индекс предыдущего, вычёркивание известного узла стоит $\Theta(1)$. Дырку в векторе не закрывают сдвигом: свободные индексы складывают в стек и выдают следующим вставкам. Живые номера при этом не меняются.

2.8. Вектор

Обычный массив неудобен фиксированной длиной. Храним буфер, число живых элементов и ёмкость. Когда места нет, выделяем буфер вдвое длиннее и копируем живые элементы.

```

1 struct Vec {
2     int* data = nullptr;
3     int n = 0;
4     int cap = 0;
5 };
6
7 void push_back(Vec& v, int x) {
8     if (v.n == v.cap) {
9         int grown = v.cap == 0 ? 1 : v.cap * 2;
10        int* fresh = new int[grown];
11        for (int i = 0; i < v.n; i++)
12            fresh[i] = v.data[i];
13        delete[] v.data;
14        v.data = fresh;
15        v.cap = grown;
16    }
17    v.data[v.n++] = x;
18 }

```

Стартовать с ёмкости 0 и умножать её на два нельзя: удвоение нуля даёт ноль. Первая ёмкость — единица, дальше степени двойки.

Теорема. n успешных добавлений в конец стоят суммарно $\Theta(n)$. Среднее время одной операции $\Theta(1)$.

Копирования случаются при ёмкостях 1, 2, 4, Сумма длин копирований меньше n . Сами записи элементов тоже линейны. Отдельное добавление бывает линейным — то, на котором случилось копирование, — но такие редки.

$$1 + 2 + 4 + \dots + 2^{k-1} = 2^k - 1 < n$$

Если ёмкость растёт в α раз при любом $\alpha > 1$, сумма копирований остаётся линейной: это геометрическая прогрессия, константа порядка $\alpha/(\alpha-1)$. Если прибавлять к ёмкости единицу, копируется $1+2+\dots+n = \Theta(n^2)$. Удвоение сразу после роста занимает половину буфера. Множитель $3/2$ держит память плотнее и копирует больше. Снятие с конца уменьшает только длину и в худшем случае $\Theta(1)$.

2.9. Стек, очередь, дек

Это обещания операций, а не картинка в памяти.

Стек умеет добавить в конец и снять с конца: кто пришёл последним, уходит первым. Очередь умеет добавить в конец и снять с начала: кто пришёл первым, уходит первым. Дек умеет все четыре операции на двух концах.

Стек прямо ложится на вектор. Очереди хватает односвязного списка, если хранить и голову, и хвост. Деку нужен двусвязный список либо кольцевой буфер.

Кольцо — вектор длины sz и два индекса. Живые элементы занимают полуинтервал от $start$ до end с переходом через нуль. Логический элемент номер i лежит в ячейке $(i + start)$ по модулю sz . Число элементов:

$$count = end - start \quad (start \leq end), \quad end - start + sz \quad (start > end)$$

Одна и та же картина $start = end$ возникает у пустого буфера и у полностью заполненного. Её различают отдельным счётчиком либо одной всегда свободной ячейкой. При удвоении элементы обходят в логическом порядке, начиная со $start$, и кладут в новый буфер подряд. Копировать массив как он лежит в памяти нельзя: логический нулевой элемент уедет.

2.10. Минимум на стеке и на очереди

Стек с минимумом — это два стека. В один кладётся значение, в другой — минимум на префиксе от дна до текущей вершины. На пустом стеке минимумов первое значение кладётся как есть. Снятие снимает оба стека. Минимум всего стека — вершина второго стека, это $\Theta(1)$ в худшем случае. Точно так же рядом ведут стек максимумов. Если минимум встретился дважды, во втором стеке лежат две копии: иначе раннее снятие сотрёт его раньше, чем из данных.

```

1 void push(Stack& data, Stack& mins, int x) {
2     data.push(x);
3     if (mins.empty()) mins.push(x);
4     else mins.push(std::min(mins.top(), x));
5 }
6
7 void pop(Stack& data, Stack& mins) {
8     data.pop();
9     mins.pop();
10 }
```

Очередь собирается из двух таких стеков. Стек b — хвост, новые элементы кладут туда. Стек a — голова, снятие идёт оттуда. Если a пуст, в него переключают b целиком: порядок переворачивается, и старое дно b оказывается началом очереди. Минимум очереди — минимум из двух вершин, пустой стек в сравнение не входит.

```

1 void push(int x) { b.push(x); }
2
3 int pop() {
4     if (a.empty()) {
5         while (!b.empty())
6             a.push(b.pop());
7     }
8     return a.pop();
9 }
10
11 int get_min() {
12     if (a.empty()) return b.min();
13     if (b.empty()) return a.min();
14     return std::min(a.min(), b.min());
15 }
```

Каждый элемент попадает в b один раз и переезжает в a не больше одного раза. Поэтому серия из m добавлений и снятий стоит суммарно $\Theta(m)$, хотя одно снятие может переложить все текущие элементы. Запрос минимума сам ничего не

перекладывает.

Один глобальный минимум, который только уменьшают при добавлении, после снятия этого элемента не знает нового ответа.

Та же идея на деке: левый стек с минимумом держит начало, правый — конец. Снятие с пустого конца переливает второй стек.

Для минимума скользящего окна кандидатов держат деком индексов. Значения возрастают от головы к хвосту, голова — минимум окна. Новый элемент выкидывает с хвоста всех, кто не меньше его и левее: в будущем окне, где новый элемент ещё жив, они минимумом не станут. С головы снимают индексы, уже выпавшие из окна. Каждый индекс входит и выходит не больше одного раза, серия из n окон стоит $\Theta(n)$. Максимум окна ведётся таким же деком с обратным неравенством.